Example Heater Controller

Software Unit Test Specification

Unit: `setpoint_ctrl`

Document ID	SUTS-SPC-001
Version	0.2
Status	In Review
Date	November 5, 2026
Unit baseline	git: 3f2a9c1 (tag v1.4.0)
Integrity level	ASIL B
Author	Jane Doe

Example Corp / Software Engineering

Document Control

Revision History

Version	Date	Author	Change
0.1	2026-10-20	Jane Doe	Initial draft
0.2	2026-11-04	Jane Doe	Fault injection test case added after review

Review and Approval

Role	Name	Signature	Date
Author	Jane Doe		
Reviewer	John Roe		
Approver	Alex Poe		

Contents

1 Introduction **3**
1.1 Purpose . . . 3
1.2 Scope . . . 3
1.3 Stakeholders and Use Cases . . . 3
1.4 References . . . 3
1.5 Terms and Abbreviations . . . 3

2 Requirements **4**
2.1 Stakeholder Requirements . . . 4
2.1.1 Functional . . . 4
2.1.2 Non-Functional . . . 4
2.2 Internal Requirements . . . 4
2.2.1 Functional . . . 4
2.2.2 Non-Functional . . . 5

3 Unit Under Test **5**
3.1 Identification . . . 5
3.2 Functional Description . . . 5
3.3 Intended Behavior . . . 5
3.4 Context . . . 6
3.5 Interfaces . . . 6
3.6 Dependencies, Stubs and Mocks . . . 6
3.7 Internal Architecture . . . 6

3.8 Algorithm Description . . . 7
3.8.1 Range Limitation . . . 7
3.8.2 Ramp Limitation . . . 7
3.9 Public API (excerpt) . . . 7

4 Test Approach **8**
4.1 Verification Methods . . . 8
4.2 Test Case Derivation . . . 8
4.3 Structural Coverage Targets . . . 9
4.4 Entry and Exit Criteria . . . 9
4.5 Pass/Fail Criteria . . . 9
4.6 Test Process . . . 9
4.7 Equivalence Classes of the Setpoint . . . 9

5 Test Environment **10**

6 Test Cases **10**
6.1 Nominal Behaviour . . . 10
6.2 Boundary Values . . . 11
6.3 Error Handling and Fault Injection . . . 11

7 Traceability **11**
7.1 Requirements to Test Cases . . . 12
7.2 Uncovered Requirements . . . 12

8 Test Results Summary **12**
8.1 Deviations . . . 13

List of Test Cases

TC-SPC-001 Initialisation sets default state . . . 10
TC-SPC-002 Setpoint accepted at upper limit . . . 11
TC-SPC-003 NvM read failure falls back to defaults . . . 11

1 Introduction

1.1 Purpose

This document specifies the unit tests for the software unit `setpoint_ctrl`. It defines the test approach, the test environment and the individual test cases used to verify that the unit fulfils its software detailed design and the allocated software requirements.

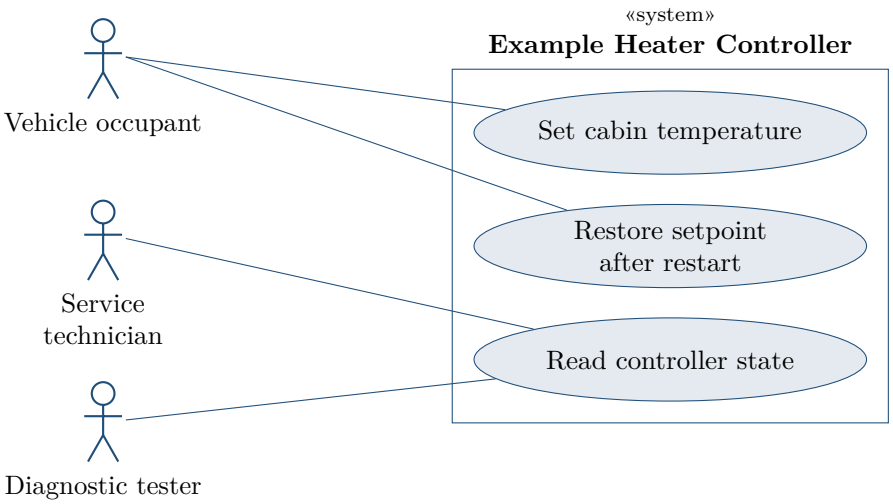
1.2 Scope

In scope: the public API of `setpoint_ctrl` (initialisation, setpoint handling, state query) in its series configuration.

Out of scope: the NvM driver and the timer HAL (tested in their own units), integration with the heater power stage (integration test).

1.3 Stakeholders and Use Cases

System-level use cases the unit contributes to, from [R1].



1.4 References

ID	Document	Type	Version
[R1]	Software Requirements Specification, SRS-SPC-001	internal	1.3
[R2]	Software Detailed Design of setpoint_ctrl, SDD-SPC-004	internal	1.1
[R3]	Software Verification Plan, SVP-PRJ-002	internal	2.0
[R4]	MISRA C:2012 Guidelines	external	2019
[R5]	ISO/IEC/IEEE 29119-3 – Test documentation	external	2021
[R6]	Automotive SPICE PAM – SWE.4 Software Unit Verification	external	4.0
[R7]	ISO 26262-6 – Clause 9, Software unit verification	external	2018

1.5 Terms and Abbreviations

Term	Definition
UUT	Unit Under Test
TC	Test Case
MC/DC	Modified Condition / Decision Coverage
NvM	Non-volatile Memory manager

2 Requirements

Stakeholder requirements state what users and other stakeholders need from `setpoint_ctrl`; internal requirements are derived from them (or from the design) and are what the unit tests verify. A stakeholder requirement counts as verified via its verified internal requirements.

2.1 Stakeholder Requirements

2.1.1 Functional

ID	Requirement	Source	Verified by
STK-010	The occupant can select a heating setpoint between 0 and 100.	Vehicle occupant, [R1]	via SWR-0110
STK-011	The selected setpoint is kept across ignition cycles.	Vehicle occupant	via SWR-0102, SWR-0120
STK-012	The service technician can read the controller state.	Service technician	via SWR-0101

2.1.2 Non-Functional

ID	Requirement	Source	Verified by
STK-020	Setpoint changes take effect smoothly, without steps in heating power.	Vehicle occupant (comfort)	*not unit tested (7.2)*
STK-021	Faults of persistent storage must not lead to an undefined heater state (ASIL B).	Safety concept	via SWR-0120

2.2 Internal Requirements

2.2.1 Functional

ID	Requirement	Derived from	Verified by
SWR-0101	After initialisation the unit is in state IDLE.	STK-012	TC-SPC-001
SWR-0102	At start-up the last setpoint is read from NvM.	STK-011	TC-SPC-001

ID	Requirement	Derived from	Verified by
SWR-0110	Setpoints outside 0..100 are rejected with E_NOT_OK; the previous setpoint is kept.	STK-010	TC-SPC-002
SWR-0120	If NvM cannot be read, the unit enters DEGRADED with setpoint 20 and reports DEM_E_NVM_READ.	STK-011, STK-021	TC-SPC-003

2.2.2 Non-Functional

ID	Requirement	Derived from	Verified by
SWR-0130	The effective setpoint changes by at most 2 K/s.	STK-020	*not unit tested (7.2)*
SWR-0140	SetpointCtrl_Main() needs at most 50 us per call on the target.	–	*not unit tested (7.2)*

3 Unit Under Test

3.1 Identification

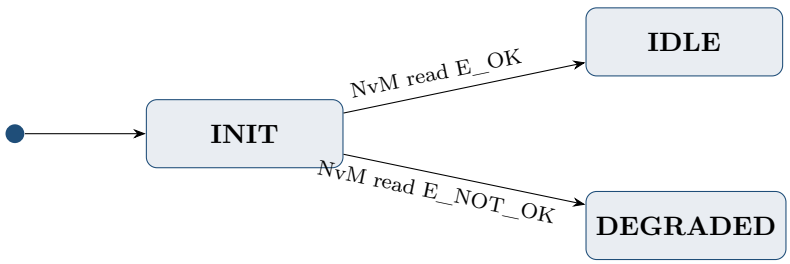
Unit name	`setpoint_ctrl`
Baseline	git: 3f2a9c1 (tag v1.4.0)
Source files	src/setpoint_ctrl.c, include/setpoint_ctrl.h
Language	C99, MISRA C:2012
Integrity level	ASIL B
Design reference	[R2], section 4.2

3.2 Functional Description

`setpoint_ctrl` stores the heater temperature setpoint, checks it against the allowed range 0–100 and restores the last valid setpoint from NvM at start-up. Details: [R2].

3.3 Intended Behavior

The unit has two operating states. Transitions happen only in `SetpointCtrl_Init()`; set requests never change the state.

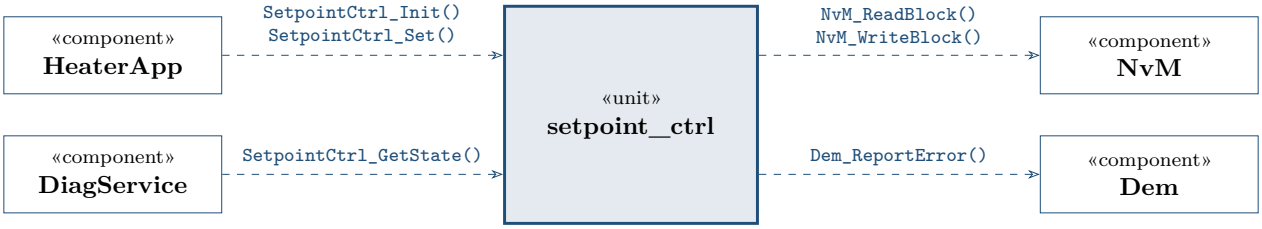


State	Setpoint	Diagnosis
IDLE	last stored value (NvM)	–
DEGRADED	default $sp_0 = 20$	`DEM_E_NVM_READ` reported once

In both states $0 \leq sp \leq 100$ holds at all times.

3.4 Context

Neighbours of `setpoint_ctrl`: callers (left) and used components (right).



3.5 Interfaces

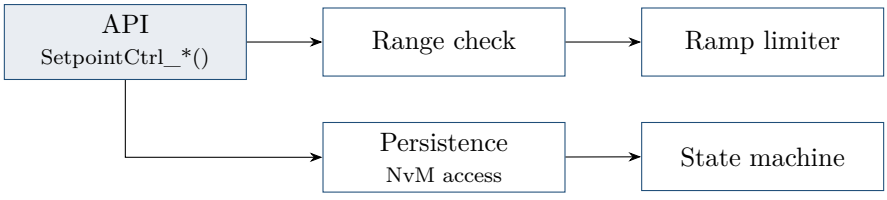
Interface	Dir.	Type / Range	Description
`SetpointCtrl_Init()`	in	void	Initialises the unit, reads NvM
`SetpointCtrl_Set(sp)`	in	uint8, [0, 100]	Sets a new setpoint, returns E_OK / E_NOT_OK
`SetpointCtrl_GetState()`	out	enum	IDLE / DEGRADED

3.6 Dependencies, Stubs and Mocks

Dependency	Replaced by	Behaviour in test
`NvM_ReadBlock()`	Mock	Returns a configurable value, records calls
`NvM_WriteBlock()`	Mock	Records written data
`Dem_ReportError()`	Mock	Records reported events

3.7 Internal Architecture

The unit consists of four internal parts; only the API functions are visible outside `setpoint_ctrl.c`.



- **Range check** – static function `sp_valid()`, implements (1).
- **Ramp limiter** – `ramp_step()`, called from `SetpointCtrl_Main()`.
- **Persistence** – wraps `NvM_ReadBlock()` / `NvM_WriteBlock()`.
- **State machine** – static `state`, see Intended Behavior.

Start-up runs through all internal parts, see Figure 1.

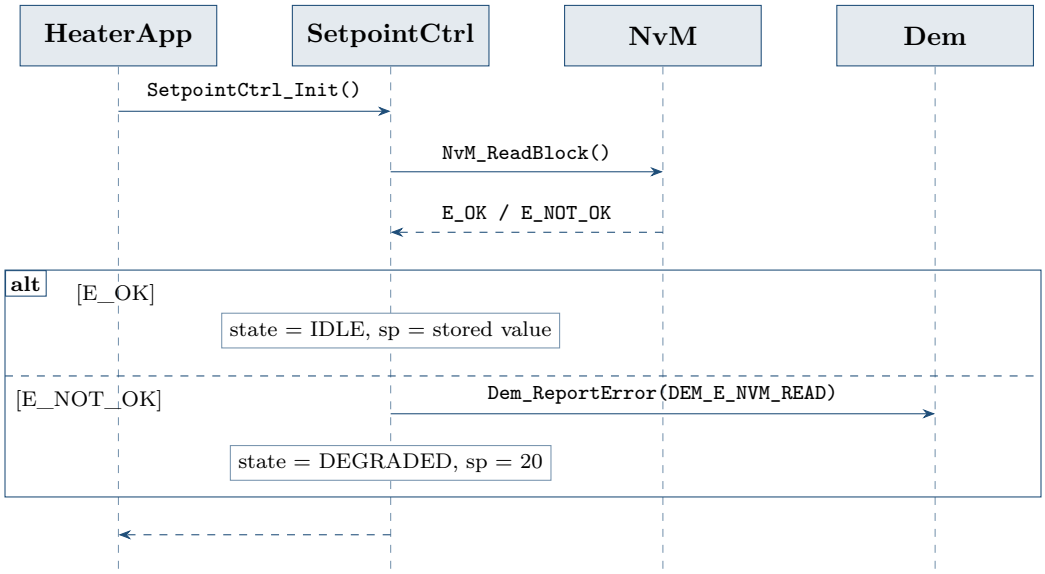


Figure 1: Start-up sequence of `SetpointCtrl_Init()`.

3.8 Algorithm Description

3.8.1 Range Limitation

A requested setpoint sp_{req} is accepted only inside the valid range:

$$sp_k = \begin{cases} sp_{\text{req}} & \text{if } 0 \le sp_{\text{req}} \le 100, \\ sp_{k-1} & \text{otherwise (request rejected, } \texttt{E_NOT_OK}\text{).} \end{cases} \tag{1}$$

3.8.2 Ramp Limitation

The effective setpoint follows the accepted one with at most $\Delta_{\max} = 2\,\text{K/s}$, sampled every $T_s = 10\,\text{ms}$:

$$\Delta_k = \text{clamp}\Big(sp_k - sp^{\text{eff}}_{k-1},\ -\Delta_{\max} T_s,\ \Delta_{\max} T_s\Big), \tag{2}$$

$$sp^{\text{eff}}_k = sp^{\text{eff}}_{k-1} + \Delta_k. \tag{3}$$

A step from 20 to 100 therefore takes $\lceil 80/(\Delta_{\max} T_s) \rceil = 4000$ cycles (40 s), see Figure 2. Equation (1) defines the expected results of TC-SPC-002; the ramp (2)–(3) (SWR-0130) is verified in integration test.

3.9 Public API (excerpt)

From `include/setpoint_ctrl.h`, copied to the unit's `code/` folder:

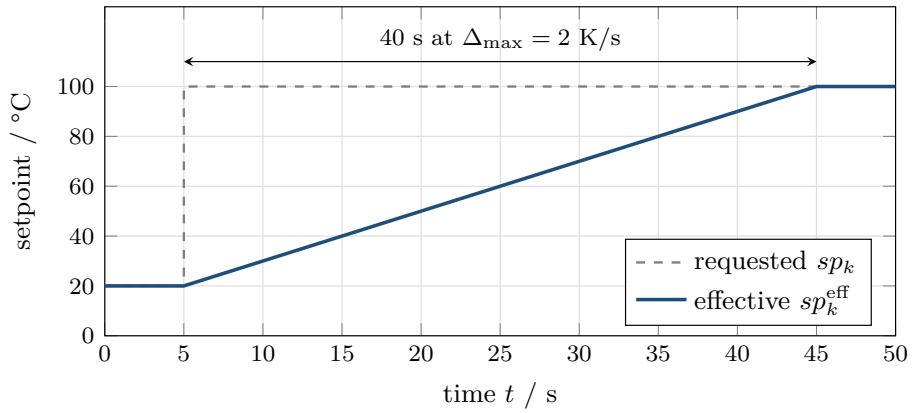


Figure 2: Ramp limitation: step request from 20 to 100 at $t = 5\,\text{s}$.

```
/* setpoint_ctrl.h -- public API of the heater setpoint controller (excerpt) */
#ifndef SETPOINT_CTRL_H
#define SETPOINT_CTRL_H

#include "Std_Types.h"

typedef enum
{
    SETPOINTCTRL_IDLE,     /* normal operation, setpoint from NvM */
    SETPOINTCTRL_DEGRADED  /* NvM unreadable, default setpoint 20 */
} SetpointCtrl_StateType;

Std_ReturnType SetpointCtrl_Init(void);
Std_ReturnType SetpointCtrl_Set(uint8 sp);   /* 0..100, else E_NOT_OK */
SetpointCtrl_StateType SetpointCtrl_GetState(void);
void SetpointCtrl_Main(void);                /* every 10 ms */

#endif /* SETPOINT_CTRL_H */
```

4 Test Approach

4.1 Verification Methods

Methods applied to this unit (ISO 26262-6 Table 7 / ASPICE SWE.4):

Method	Applied	Remark
Requirements-based test	☒	
Interface test	☒	
Fault injection test	☒	NvM failures
Resource usage evaluation	☐	done at integration level
Back-to-back test (model vs. code)	☐	no model

4.2 Test Case Derivation

Test cases are derived using:

- analysis of requirements and detailed design
- equivalence class partitioning of inputs and states
- boundary value analysis (min, max, max+1)
- error guessing

4.3 Structural Coverage Targets

Metric	Target	Justification if not met
Statement	100 %	review of uncovered lines
Branch	100 %	review of uncovered branches
MC/DC	n.a.	not required for ASIL B

4.4 Entry and Exit Criteria

Entry:

- Unit source is baselined (git: 3f2a9c1 (tag v1.4.0)) and compiles without warnings.
- Requirements [R1] and design [R2] are reviewed.

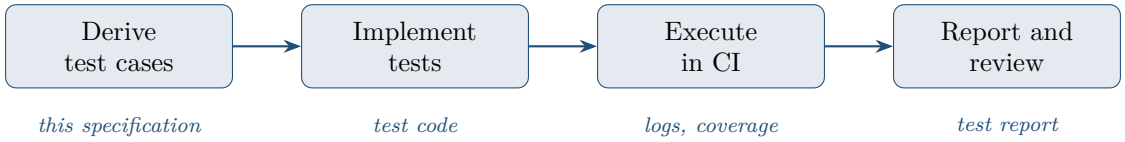
Exit:

- All test cases executed; all passed or deviations accepted.
- Coverage targets reached or gaps justified.

4.5 Pass/Fail Criteria

A test case passes if every step yields the expected result and no unexpected side effect (assertion, sanitizer report, timeout) occurs.

4.6 Test Process



4.7 Equivalence Classes of the Setpoint

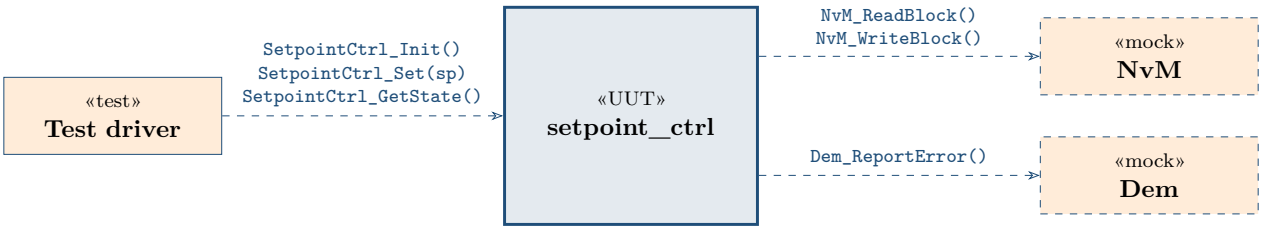
Class	Range	Representatives (boundaries)
invalid low	$sp < 0$	not representable (`uint8`)
valid	$0 \le sp \le 100$	0, 1, 50, 99, 100
invalid high	$100 < sp \le 255$	101, 255

Statement coverage is measured as

$$C_{\text{stmt}} = \frac{\text{executed statements}}{\text{all statements}} \cdot 100\,\%.$$

5 Test Environment

Test harness: the test driver calls the interfaces of the UUT; its dependencies are replaced by test doubles.



Item	Description	Version
Execution platform	Host x86_64 (Linux)	
Compiler / flags	gcc `-O0 -g --coverage`	14.2
Test framework	GoogleTest	1.15
Coverage tool	gcov + lcov	2.1

Tool confidence level TCL1 for all tools (ISO 26262-8 clause 11); no qualification needed.

6 Test Cases

6.1 Nominal Behaviour

TC-SPC-001 Initialisation sets default state

Objective	SetpointCtrl_Init() sets all internal state to the documented defaults.
Requirements	SWR-0101, SWR-0102
Method	Requirements-based
Priority	High
Preconditions	Unit not initialised; NvM mock returns E_OK with setpoint 42.
Input data	none

Test steps

#	Action / Stimulus	Expected result
1	Call SetpointCtrl_Init().	Returns E_OK.
2	Call SetpointCtrl_GetState().	Returns IDLE.
3	Check the NvM mock call log.	NvM_ReadBlock() called exactly once.

Postconditions	Unit in IDLE, setpoint = 42.
Implementation	test/setpoint_ctrl_test.cpp: InitTest.SetsDefaults

Comparison of real values. Where a step expects a computed value y, it passes if

$$|y - y_{\text{exp}}| \le \varepsilon, \qquad \varepsilon = 10^{-3} \cdot \max(1, |y_{\text{exp}}|).$$

6.2 Boundary Values

TC-SPC-002 Setpoint accepted at upper limit

Objective	Range check of the setpoint at its upper bound.
Requirements	SWR-0110
Method	Boundary value analysis
Priority	High
Preconditions	Unit initialised (TC-SPC-001).
Input data	`sp` 100 max (valid) `sp` 101 max+1 (invalid)

Test steps

#	Action / Stimulus	Expected result
1	SetpointCtrl_Set(100)	Returns E_OK; setpoint = 100.
2	SetpointCtrl_Set(101)	Returns E_NOT_OK; setpoint unchanged (100).

Postconditions	Setpoint = 100.
Implementation	test/setpoint_ctrl_test.cpp: RangeTest.UpperLimit

6.3 Error Handling and Fault Injection

All fault injection tests use the NvM mock in *error* mode; see [R3] for the mock configuration.

TC-SPC-003 NvM read failure falls back to defaults

Objective	Defined behaviour when the persistent setpoint cannot be read.
Requirements	SWR-0120
Method	Fault injection; error guessing
Priority	Medium
Preconditions	NvM mock configured to return E_NOT_OK.

Test steps

#	Action / Stimulus	Expected result
1	Call SetpointCtrl_Init().	Returns E_OK.
2	Call SetpointCtrl_GetState().	Returns DEGRADED.
3	Check the Dem mock.	Event DEM_E_NVM_READ reported once.

Postconditions	Unit degraded, default setpoint 20.
Implementation	test/setpoint_ctrl_test.cpp: NvmTest.ReadFailure

7 Traceability

7.1 Requirements to Test Cases

Requirement	Type	Description	Test cases
STK-010	stakeholder, functional	The occupant can select a heating setpoint between 0 and 100.	via SWR-0110
STK-011	stakeholder, functional	The selected setpoint is kept across ignition cycles.	via SWR-0102, SWR-0120
STK-012	stakeholder, functional	The service technician can read the controller state.	via SWR-0101
STK-020	stakeholder, non-functional	Setpoint changes take effect smoothly, without steps in heating power.	*none – see 7.2*
STK-021	stakeholder, non-functional	Faults of persistent storage must not lead to an undefined heater state (ASIL B).	via SWR-0120
SWR-0101	internal, functional	After initialisation the unit is in state IDLE.	TC-SPC-001
SWR-0102	internal, functional	At start-up the last setpoint is read from NvM.	TC-SPC-001
SWR-0110	internal, functional	Setpoints outside 0..100 are rejected with E_NOT_OK; the previous setpoint is kept.	TC-SPC-002
SWR-0120	internal, functional	If NvM cannot be read, the unit enters DEGRADED with setpoint 20 and reports DEM_E_NVM_READ.	TC-SPC-003
SWR-0130	internal, non-functional	The effective setpoint changes by at most 2 K/s.	*none – see 7.2*
SWR-0140	internal, non-functional	SetpointCtrl_Main() needs at most 50 us per call on the target.	*none – see 7.2*

7.2 Uncovered Requirements

Requirements that no test case verifies (directly or, for stakeholder requirements, via internal ones):

Requirement	Rationale / verified instead by
STK-020	via derived SWR-0130 – see their rationale
SWR-0130	Timing behaviour, verified in integration test ITS-SPC-007
SWR-0140	Target timing measurement, see timing analysis report TAR-SPC-001

8 Test Results Summary

Executed on 2026-11-03 in CI pipeline run 1234.

Test cases specified	3
Executed / Passed / Failed	3 / 3 / 0
Statement coverage	100 %
Branch coverage	95.5 %
Deviations	1

8.1 Deviations

ID	Test case	Description / Justification	Status
DEV-01	TC-SPC-002	Branch for sp > 255 unreachable (uint8); accepted by review.	closed